Product predictor: Random Forest (Vishal)

Predicts whether an Amazon product will be a **top-20% seller in its own category** (the team's shared target), using a Random Forest. It's compared head-to-head with my teammate's XGBoost on the same products, target and inputs.

Results

Tested on the teammate's split: 249,857 test products, about 1M training products. Both models use the teammate's title score and brand rate. Higher is better on every score.

Model	**ROC-AUC**	**PR-AUC**	**ROC-AUC within category**
Baseline: category only	0.693	0.319	0.500
Baseline: title score alone	0.769	0.416	0.718
Random Forest, full (no isBestSeller)	**0.834**	**0.537**	**0.797**
Random Forest, launch-time only	**0.801**	**0.476**	**0.748**
Teammate's XGBoost, full	0.850	0.580	0.810
Teammate's XGBoost, launch-time	0.810	0.500	0.760

Takeaways:

- **XGBoost is slightly ahead on every score:** about 0.01–0.02 ROC-AUC and 0.02–0.04 PR-AUC.
- **Most of XGBoost's earlier lead came from the title features, not the algorithm.** Before they were added, the RF scored 0.81 / 0.49 / 0.75 (full) and 0.75 / 0.40 / 0.65 (launch-time).
- The Random Forest hasn't been tuned yet.

How to run

1. Put these in `data/` next to the notebook (they're not in git because they're too big):
   - `amazon_products.csv` and `amazon_categories.csv` from the Kaggle dataset
   - `split_for_vishal.csv` (teammate's train/test split). Without it, the notebook falls back to a 300k random sample.
2. Create a Python environment and install `requirements.txt`. In VS Code: Cmd/Ctrl+Shift+P → **Python: Create Environment** → Venv.
3. Open `marketlens_random_forest.ipynb`, select the `.venv` kernel and click **Run All**. It takes about 10–20 minutes on a laptop.

All settings are in the Setup cell: `SPLIT_FILE`, `USE_TITLE_FEATURES`, `MIN_SAMPLES_LEAF`, `RUN_APPENDIX` and `TEAMMATE_SCORES`.

What's in the notebook

1. Load the data, clean it, and check outliers.
2. Look at how sales spread across categories, including an opportunity map.
3. Features and the target. Label rule: sales at or above the category's 80th percentile **and** above 0.
4. Leak-safe encodings.
   - Category average and title score/brand rate are built 5-fold out-of-fold on the training rows.
   - Test rows only ever see models and averages fit on the training rows.
   - `isBestSeller` is excluded, because Amazon sets it from sales.
5. Random Forest in two versions (full and launch-time only), compared against four baselines and the teammate's XGBoost.
6. Charts saved in `results/`. The main one is `rf_vs_xgboost.png`.
7. Appendix: version 1, which predicted log monthly sales and was scored with R^2. It's off by default.

Caveats

- `brand_rate` assumes the brand already sells on Amazon. A brand-new brand just gets the average.
- Variants of the same product (different sizes or colors) can land in both train and test, which may flatter the title and brand features. This affects both models equally.